

# Taller Práctico: Memoria y Archivos

Malak Sanchez

Monitorías de Sistemas Operativos

## Introducción

El siguiente taller tiene como objetivo integrar los conceptos fundamentales vistos en el Capítulo 0: paso de parámetros por línea de comandos, gestión de memoria dinámica y manipulación de archivos. Los problemas planteados requieren un análisis detallado de la estructura de datos y una gestión responsable de los recursos del sistema.

## Problema 1: Generador Dinámico de Patrones de Marco

Diseñe un programa que genere una matriz “marco” en memoria dinámica. El programa debe recibir por línea de comandos dos enteros positivos  $N$  y  $M$ , y un carácter  $C$ .

**Ejemplo de ejecución:**

```
$ ./pattern_gen 5 5 #
```

**Ejemplo de salida:**

```
# # # # #
# . . . #
# . . . #
# . . . #
# # # # #
```

**Requerimientos:**

1. El programa debe validar que se pasen los 3 argumentos necesarios.
2. Debe reservar memoria dinámica para una matriz de caracteres (`char **`) de tamaño  $N \times M$ .
3. Rellene los bordes de la matriz con el carácter  $C$  y el interior con puntos (`.`).
4. Implemente una función `print_matrix` y una función `free_matrix` para gestionar la limpieza.
5. El programa debe imprimir la matriz en pantalla.
6. La matriz creada debe ser dinámica, es decir, debe poder aceptar cualquier valor de  $N$  y  $M$  e imprimir correctamente el patrón.

## Problema 2: El Analizador de Bitácoras (Log Parser)

Diseñe una herramienta que procese un archivo de texto con registros de eventos de un sistema. La primera fila del archivo contiene un valor  $n$  que nos dice la cantidad de elementos a leer. Cada línea del archivo representa un evento con el formato:

```
<ID_PROCESO> <TIMESTAMP> <CONSUMO_MEMORIA_MB>
```

**Ejemplo de comando:**

```
$ ./log_parser input.txt 100
```

**Ejemplo de salida esperada (Consumo superior a 100 MB):**

```
1024 1710432000 101.5
1025 1710432001 120.0
1026 1710432002 200.0
```

**Requerimientos:**

1. Debe leer el archivo y almacenar los datos correctamente.
2. Debe usar `fscanf(file, "%d%f%f", &id, &timestamp, &memory_consumption)` para leer los datos del archivo con el formato especificado.
3. Filtre y muestre solo los procesos cuyo consumo de memoria sea superior al umbral pasado por parámetro.
4. Cuente la cantidad de procesos que consumen más memoria que el umbral e imprima el resultado.

## Problema 3: Mini-Compresor de Texto RLE

Implemente un programa que reciba como parametros un archivo *input.in* y realice una compresión por longitud de caracteres (RLE - Run-Length Encoding).

**Ejemplo de comando:**

```
$ ./rle_compress input.in
```

**Ejemplo visual de funcionamiento:**

```
Entrada (en archivo): AAAAABBBCC  
Salida (consola): 5A3B2C
```

**Requerimientos:**

1. Cargue el contenido del archivo pasado como argumentos del main.
2. El programa debe validar que el archivo de entrada no esté vacío antes de proceder.
3. Almacene la cadena comprimida en un *buffer*.
4. Imprima en pantalla el “Ratio de Compresión” (Tamaño Original / Tamaño Comprimido).
5. Libere correctamente la memoria dinámica utilizada.

## Criterios de Evaluación

- **Gestión de Memoria:** Uso correcto de `malloc/free` y ausencia de fugas.
- **Robustez:** El programa no debe fallar si el archivo no existe o los argumentos son insuficientes.
- **Claridad:** Código bien estructurado y documentado en lugares clave.